SEMINAR · NHẬP MÔN TƯ DUY THUẬT TOÁN

Trie

Cây tiền tố

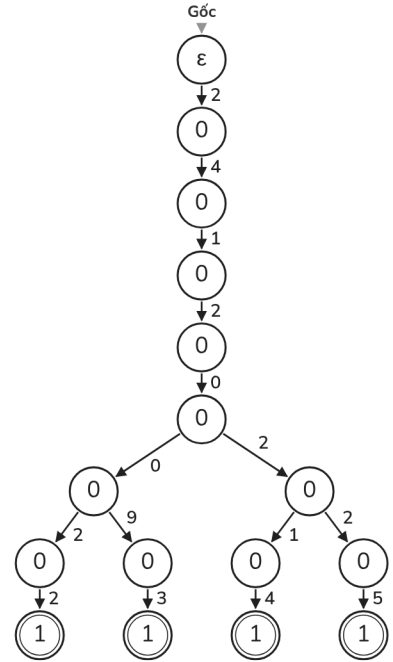
Tổ chức dữ liệu theo tiền tố chung.

24120093 Lê Quang Minh

24120022 Hà Văn Thiên Bảo

24120225 Trịnh Văn Thành

24120214 Nguyễn Ngọc Phúc



Nội dung trình bày

01 Đặt vấn đề và ý tưởng Trie
02 Cài đặt và thao tác cơ bản
03 Độ phức tạp và bộ nhớ
04 So sánh trên cùng bài toán
05 Điểm mạnh, hạn chế và lựa chọn
06 Bài toán ứng dụng và mở rộng
07 Ứng dụng trong hệ thống thực tế
08 Tổng kết

PHẦN I

01 Đặt vấn đề và ý tưởng Trie

Từ bài toán tìm từ đến cách tổ chức dữ liệu theo tiền tố.

Tìm từ trong một danh sách lớn

`app` `apple` `ape` `bat` `bath`

- Có từ `apple` không?
- Bao nhiêu từ bắt đầu bằng `ap`?

Duyệt tuyến tính

Duyệt từng từ → so sánh → tìm / đếm.

Truy vấn	Chi phí xấu nhất
Tìm từ	$O(nL)$
Đếm tiền tố	$O(n\|p\|)$

n: số từ · L: độ dài tối đa · |p|: độ dài tiền tố (≤ L).

Tra từ điển bằng cách thu hẹp tiền tố

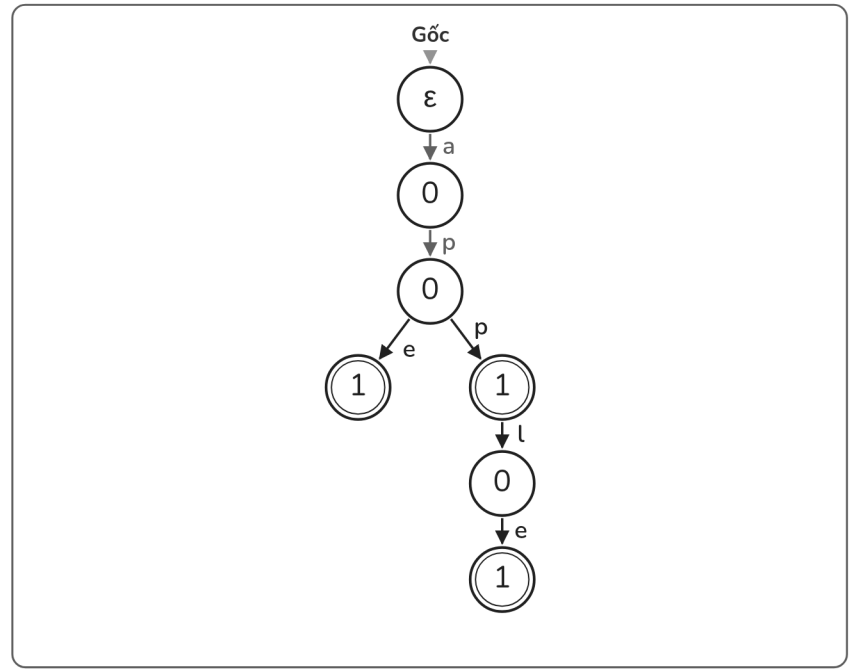


Mỗi ký tự thu hẹp nhóm từ cần xét.

Các từ chung phần đầu có thể được gom lại.

Trie biểu diễn tiền tố như thế nào?

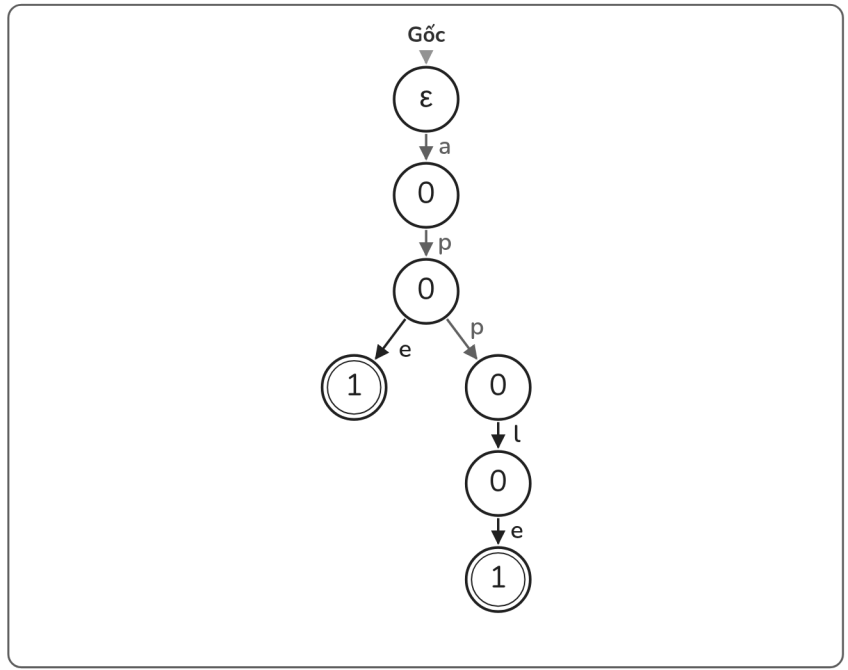
- Mỗi cạnh mang **một ký tự**.
- Đường đi từ gốc tạo thành **một tiền tố**.
- Gốc là xâu rỗng.
- Từ chung phần đầu dùng chung đường đi.



Có đường đi chưa chắc có từ

Chỉ chèn `apple` và `ape`.

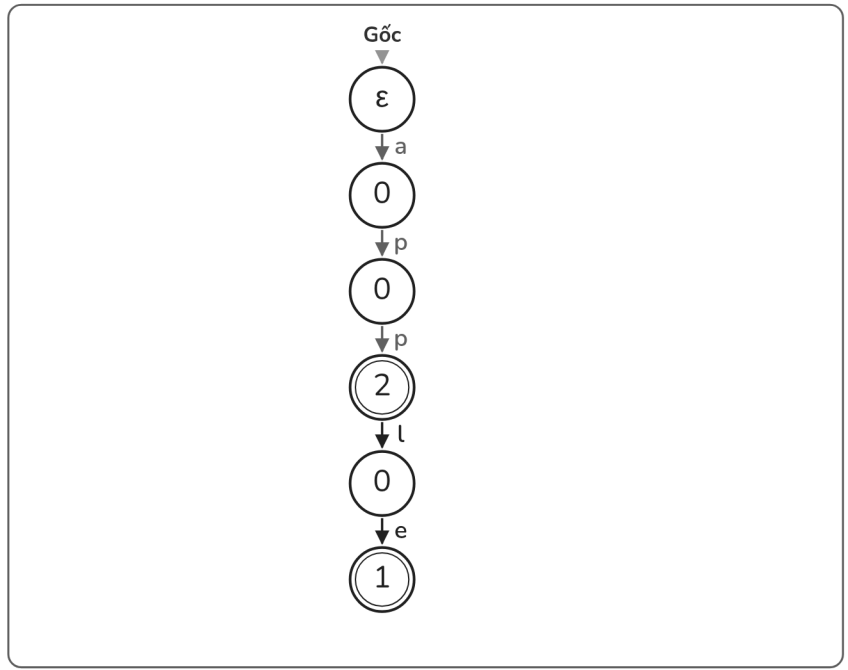
Xâu	Có đường	Đã chèn
apple	✓	✓
ape	✓	✓
app	✓	**Không**



Đánh dấu từ bằng endCount

`endCount[u]`: số lần một từ kết thúc tại node u.

- Có từ hoàn chỉnh khi **endCount[u] > 0**.
- Chèn `app` hai lần → **endCount[app] = 2**.
- Node kết thúc từ vẫn có thể có con.



PHẦN II

02 Cài đặt và thao tác cơ bản

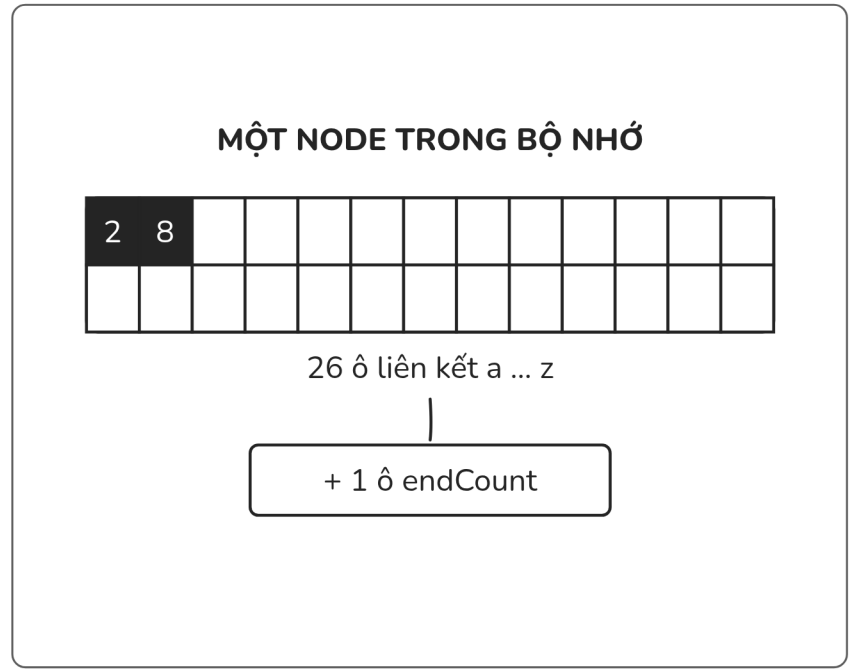
Biểu diễn Trie · Chèn · Tìm · Xóa

Lưu Trie bằng mảng hai chiều

```
int trie[MAXN][26];
int endCount[MAXN];
int nodeCount = 1; // root = 1
```

`trie[u][c]`: chỉ số node con.

0 = thiếu cạnh.

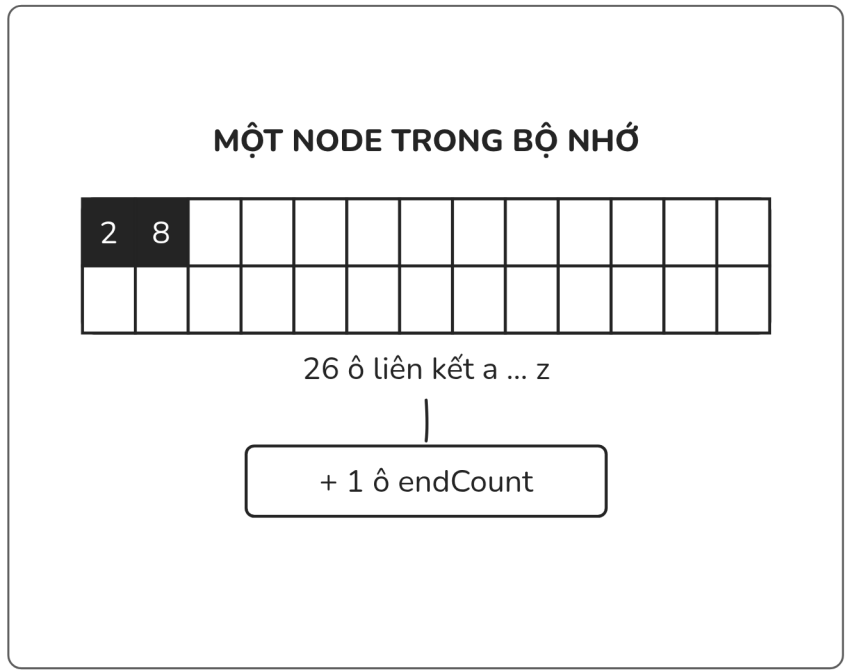


Lưu Trie bằng node và con trỏ

```
struct Node {
    Node* child[26]{};
    int endCount = 0;
};
```

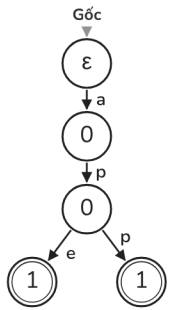
`child[c]` : con trỏ đến node con.

nullptr = thiếu cạnh.



Chèn từ

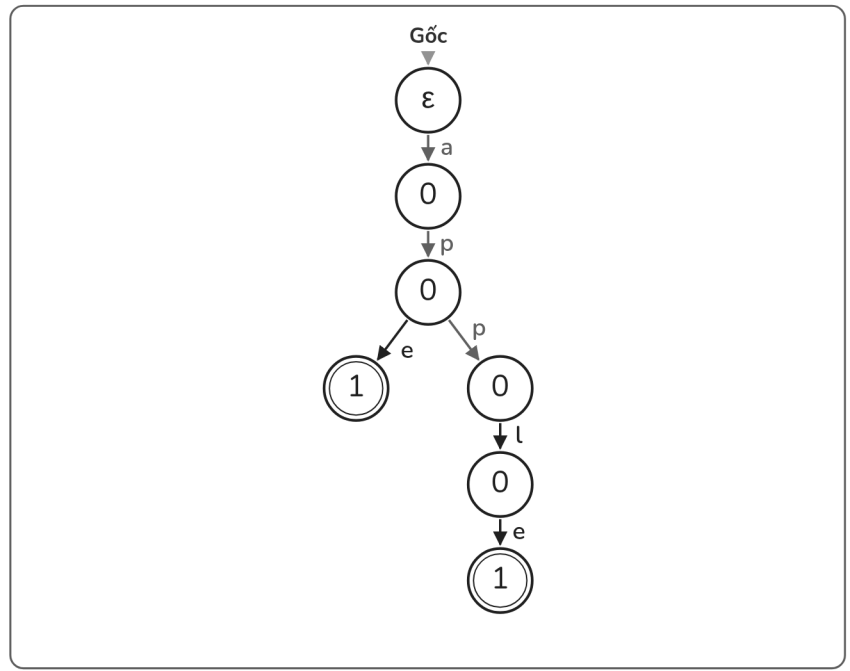
```
void insert(const string& s) {
    int u = 1;
    for (char ch : s) {
        int c = ch - 'a';
        if (!trie[u][c])
            trie[u][c] = ++nodeCount;
        u = trie[u][c];
    }
    ++endCount[u];
}
```



Chèn apple.

Tìm node theo một xâu

```
int findNode(const string& s) {
    int u = 1;
    for (char ch : s) {
        u = trie[u][ch - 'a'];
        if (u == 0) return 0;
    }
    return u;
}
```



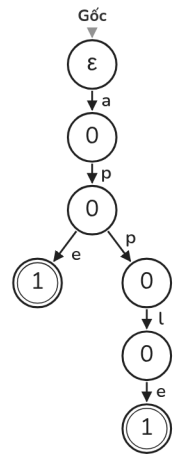
Kiểm tra tiền tố: có đường đi không?

Với `u = findNode(p)`:

> StartsWith: u != 0

Đã chèn `apple`, `ape`.

`app` là tiền tố → **đúng**.



Kiểm tra tiền tố app.

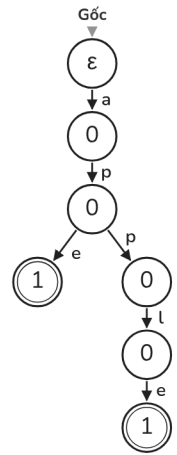
Tìm từ: có điểm kết thúc không?

Với `u = findNode(s)`:

> Search: u && endCount[u] > 0

Đã chèn `apple`, `ape`.

`app` có đường đi nhưng **chưa phải từ đã lưu**.



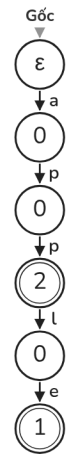
Tìm từ app.

Xóa một lần xuất hiện

```
void eraseOne(const string& s) {
    int u = findNode(s);
    if (u && endCount[u] > 0)
        --endCount[u];
}
```

Giảm `endCount`.

Giữ các node.

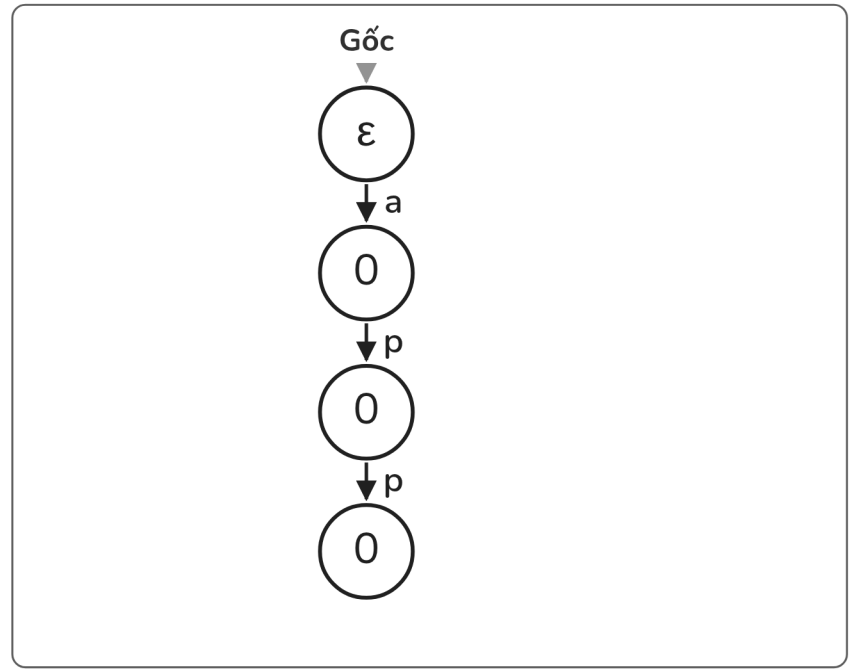


Xóa từng lần xuất hiện của app.

Sau xóa, đường cũ có thể vẫn còn

Chèn `app`, rồi xóa lần xuất hiện duy nhất.

- `endCount[app] = 0` → Search trả **sai**.
- Đường `a → p → p` vẫn còn.
- StartsWith cần **bộ đếm đi qua** hoặc **dọn nhánh**.

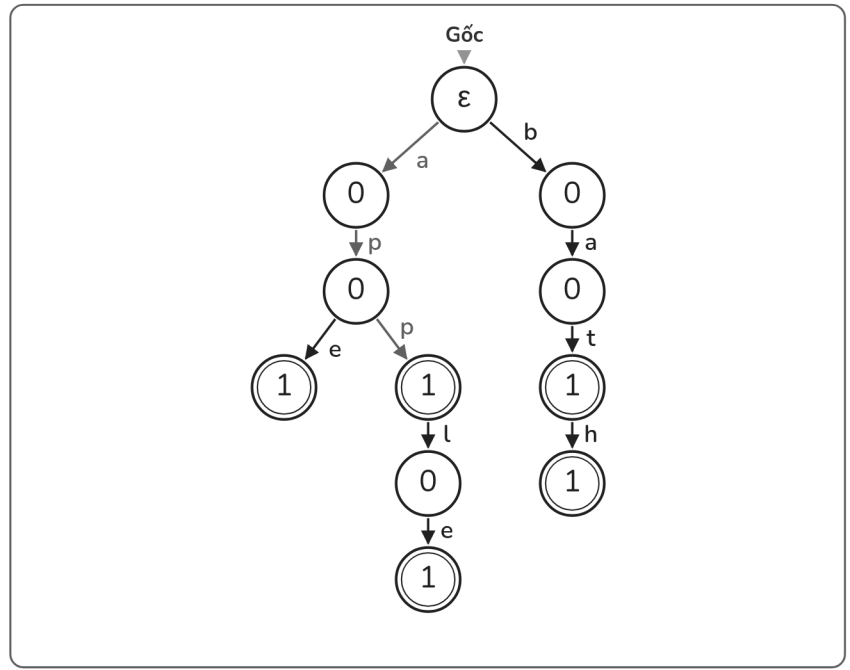


Điểm chung: duyệt một đường theo ký tự

Gốc → **a** → **p** → **p**

Mỗi ký tự chọn một cạnh.

Không cần xét từng từ riêng lẻ.



PHẦN III

03 Độ phức tạp và bộ nhớ

Thời gian truy vấn · Số node · Chi phí lưu trữ

Thời gian phụ thuộc độ dài xâu

Thao tác	Thời gian
Chèn / tìm / kiểm tra tiền tố / xóa logic	$O(L)$
Xây Trie từ toàn bộ dữ liệu	$O(S)$

$$S = \sum_{i} |s_i| \le nL$$

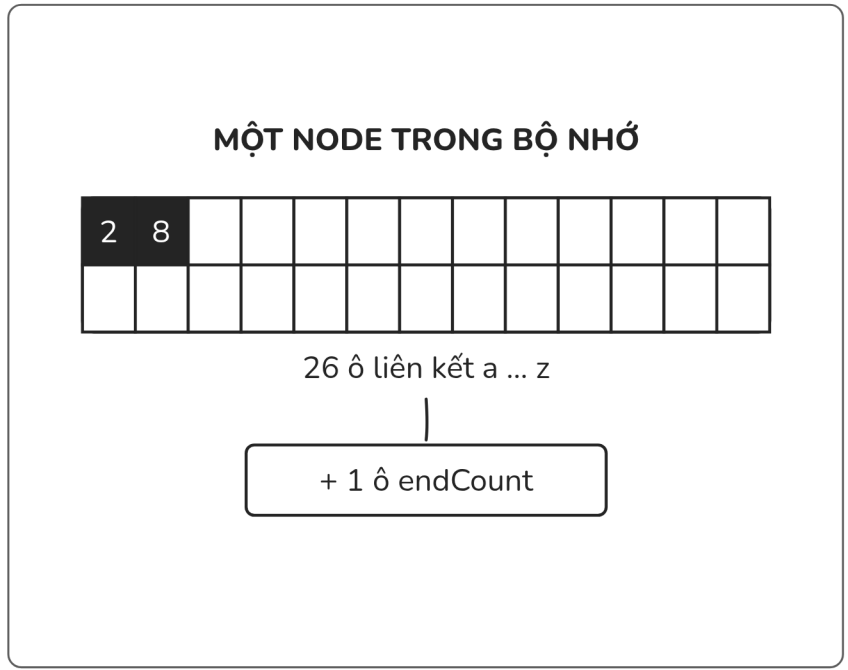
Từ số node đến bộ nhớ cấp phát

N node, gồm gốc. Gốc đánh số 1.

MAXN ≥ N + 1 vì chỉ số 0 dành cho thiếu cạnh.

Mảng `trie` và `endCount` chiếm:

> **MAXN × 27 × sizeof(int) byte**



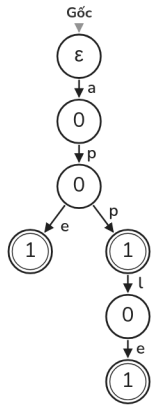
Cần tối đa bao nhiêu node?

Mỗi ký tự thêm tối đa **1 node**.

$$N \leq 1 + nL$$

n từ · dài tối đa L ký tự · N gồm gốc

Ví dụ: 1.000 từ, tối đa 20 ký tự/từ → **N ≤ 20.001**.



Cận: 1 + 3 × 5 = **16 node**
Đếm trên cây: **7 node**

Chèn app, apple, ape.

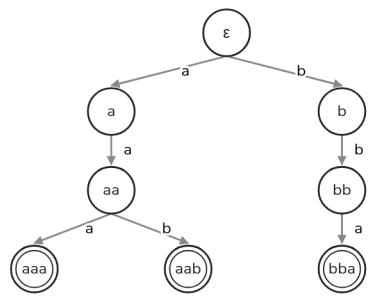
Đếm node theo từng tầng

Tầng d có tối đa **min(n, σ^d)** node.

$$N_{\max} = 1 + \sum_{d=1}^{L} \min(n, \sigma^d)$$

σ: số ký tự · n: số từ · L: độ dài tối đa.

3 từ: aaa, aab, bba · σ = 2 · L = 3



Tầng 1	$\min(3, 2^1) = 2$	—
Tầng 2	$\min(3, 2^2) = 3$	—
Tầng 3	$\min(3, 2^3) = 3$	—

Bắt đầu từ 1 node gốc.

Ví dụ: 1.000 từ dài tối đa 20 ký tự

Bảng chữ cái có **26** ký tự.

Tầng	Số node tối đa
1	26
2	676
3–20	18 × 1.000
Tổng, gồm gốc	**18.703**

Cận đơn giản **20.001** → cận theo tầng **18.703**.

PHẦN IV

04 So sánh trên cùng bài toán

Đối chiếu Trie với duyệt tuyến tính, mảng, set và bảng băm.

Bốn kiểu truy vấn

A **Tìm từ**

Có `apple` không?

B **Kiểm tra tiền tố**

Có từ bắt đầu bằng `app`?

C **Đếm**

Bao nhiêu từ bắt đầu bằng `app`?

D **Liệt kê**

Lấy k từ đầu theo thứ tự từ điển.

So sánh truy vấn tìm từ chính xác

Cách lưu	Tìm từ dài L
Mảng chưa sắp xếp	$O(nL)$
Mảng sắp xếp + binary search	$O(L \log n)$
`std::set<string>`	$O(L \log n)$
`std::unordered_set<string>`	$O(L)$ kỳ vọng
Trie 26 nhánh	$O(L)$

n: số từ · L: độ dài từ · Các cận xấu nhất, riêng hash là kỳ vọng.

So sánh truy vấn kiểm tra tiền tố

Cách lưu	Tiền tố dài p
Mảng chưa sắp xếp	$O(np)$
Mảng sắp xếp + binary search	$O(p \log n)$
`std::set<string>`	$O(p \log n)$
`std::unordered_set<string>`	Không trực tiếp. Quét $O(np)$
Trie 26 nhánh	$O(p)$

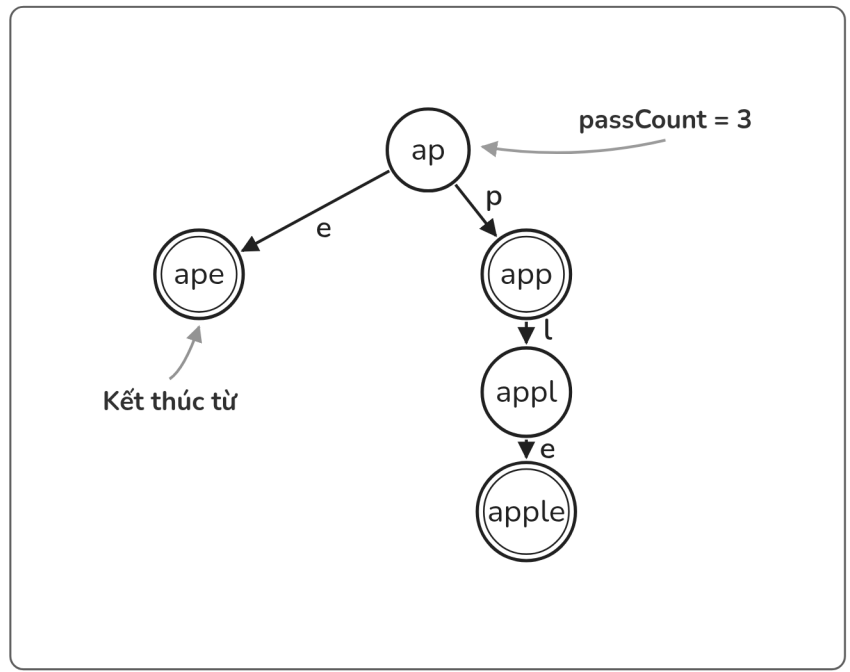
n: số từ · p: độ dài tiền tố · Trie cần bộ nhớ node.

So sánh truy vấn đếm tiền tố

Tiền tố dài p:

- Trie + `passCount`: **O(p)**.
- Mảng sắp xếp: tìm hai biên, **O(p log n)**.

passCount là thống kê bổ sung cho bài toán đếm.

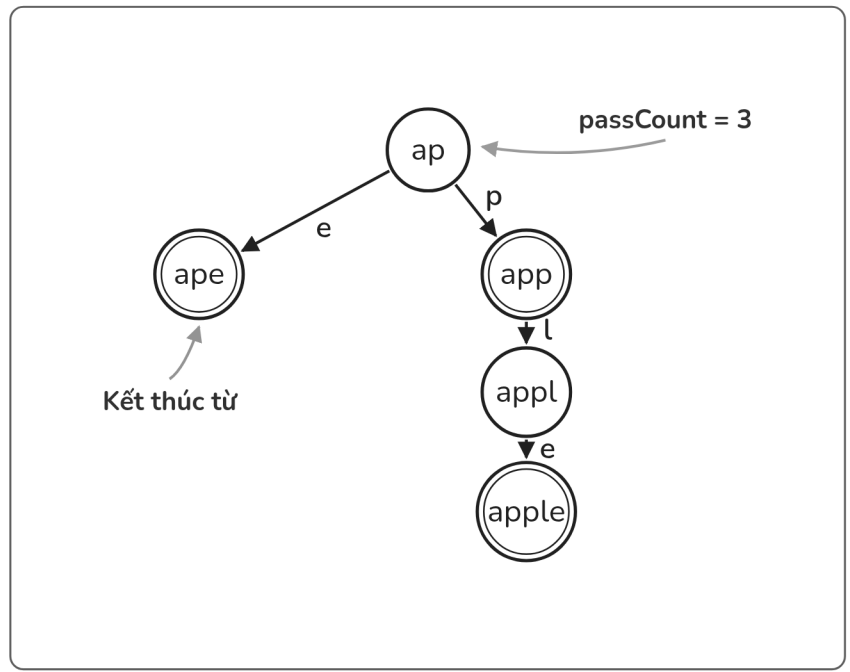


Liệt kê gợi ý còn có chi phí đầu ra

Đến node tiền tố → duyệt cây con → xuất từ.

O(p) chỉ là chi phí tìm node tiền tố.

Còn thời gian duyệt các node và xuất k kết quả.



Chỉ tìm khóa hoặc cần thứ tự?

Nhu cầu	Cấu trúc nên cân nhắc
Chỉ tìm từ chính xác	`std::unordered_set<string>`
Cần phần tử trước / sau	`std::set<string>` / cây cân bằng

Chọn theo thao tác thực sự cần hỗ trợ.

Dữ liệu tĩnh: mảng sắp xếp là một lựa chọn

Sắp xếp một lần → tìm / đếm tiền tố nhiều lần.

Mảng sắp xếp + binary search.

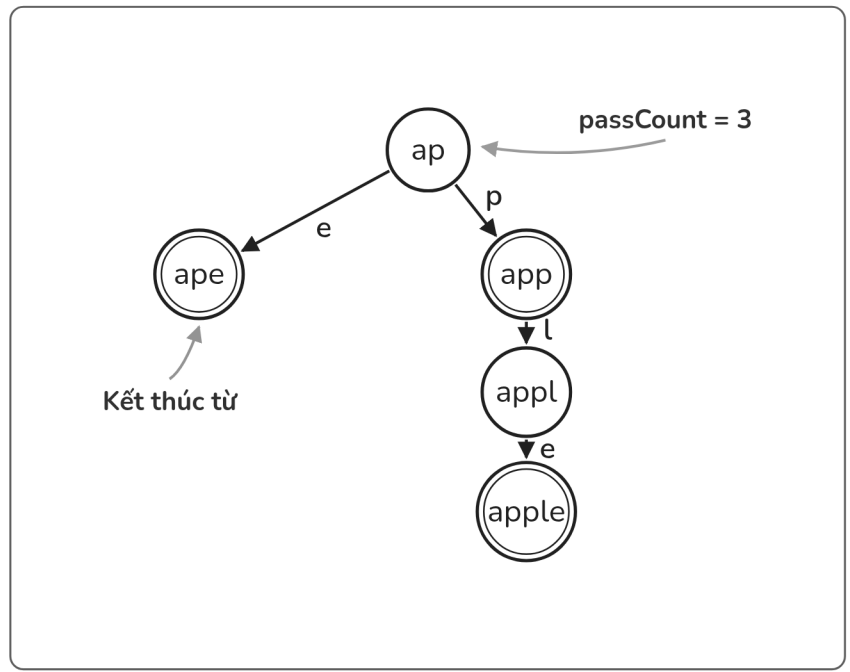
- Không cần cấp node cho từng tiền tố.
- Cập nhật chèn / xóa thường tốn hơn.

Nhiều truy vấn tiền tố kèm cập nhật

Cân nhắc Trie khi cần:

- Chèn / xóa thường xuyên.
- Tìm hoặc đếm theo tiền tố.
- Cập nhật thống kê theo đường đi.

Vẫn phải kiểm tra giới hạn bộ nhớ và bảng chữ cái.



PHẦN V

05 Điểm mạnh, hạn chế và lựa chọn

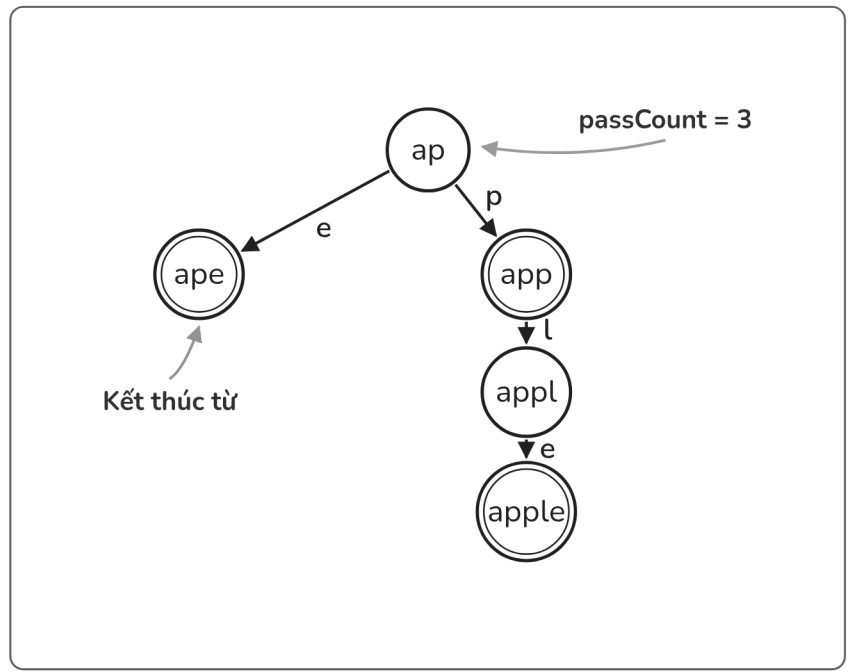
Nhận diện bài toán phù hợp và các chi phí cần cân nhắc.

Điểm mạnh: gom từ theo tiền tố

Các từ cùng tiền tố nằm trong **một cây con**.

Đến tiền tố dài p chỉ cần **O(p)**.

Từ đó có thể xử lý cả nhóm từ liên quan.

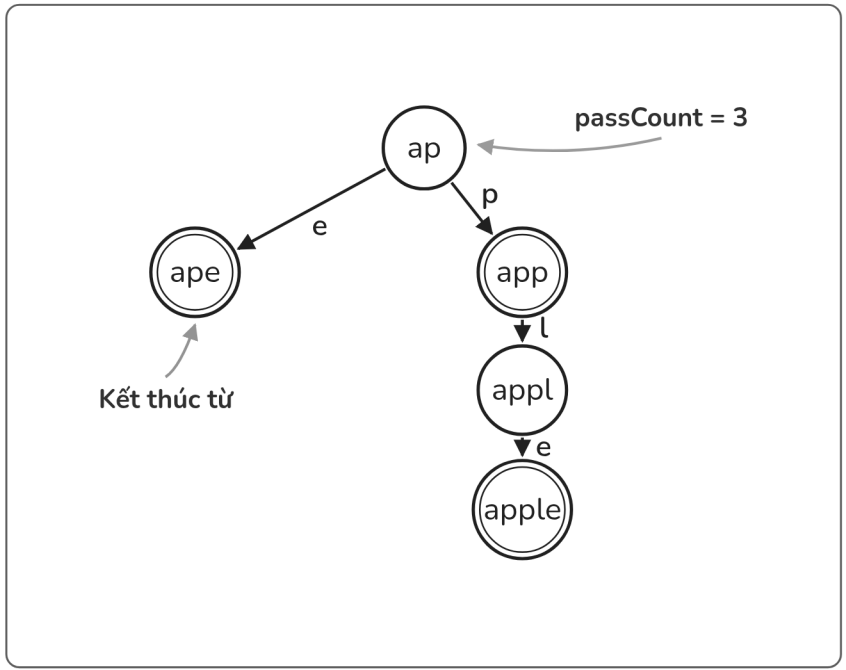


Gắn thêm thống kê lên node

Mỗi node có thể lưu:

- Tần suất xuất hiện.
- Trọng số.
- Số từ trong cây con.

Thêm thống kê → thêm bộ nhớ và công cập nhật.

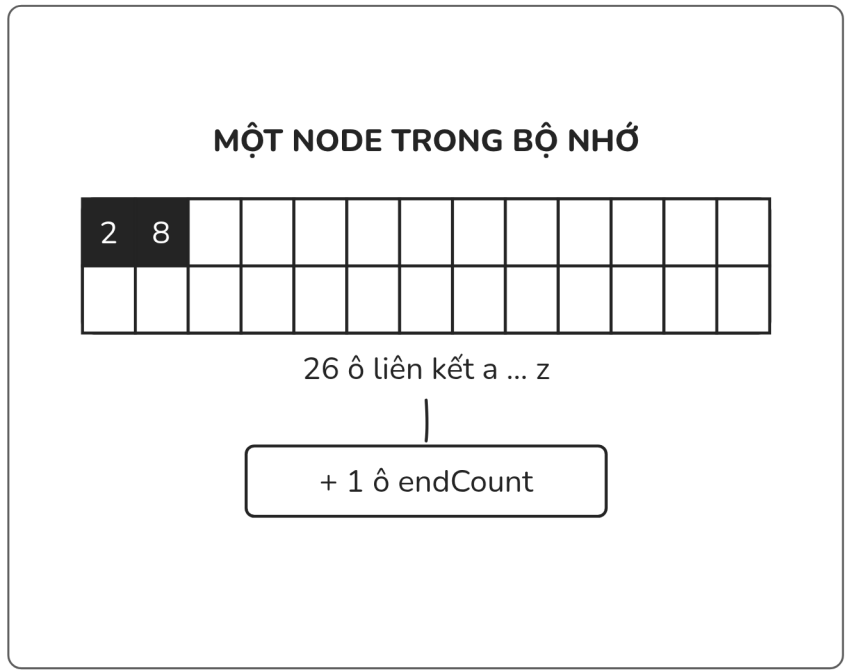


Hạn chế: nhiều ô liên kết không dùng

Một node có **26 ô**, dù chỉ có một con.

Mảng tĩnh chiếm bộ nhớ theo **MAXN**.

Cần tính cả chi phí **cache / cấp phát**.



Bảng chữ cái lớn cần cách lưu khác

`child[26]` chỉ phù hợp với **a–z**.

Unicode hoặc tập ký hiệu lớn:

Cần cách ánh xạ và lưu nhánh phù hợp.

Tiền tố khác chuỗi con

`apple`

- `app` : bắt đầu từ gốc → **tiền tố**.
- `ple` : nằm giữa từ → **chuỗi con**.

Trie cơ bản không tự giải tìm chuỗi con bất kỳ.

-> Giải quyết bằng thuật toán Aho-corasick.

Khi nào dùng Trie?

Nên cân nhắc

- Dữ liệu là dãy ký hiệu / bit.
- Nhiều truy vấn tiền tố.
- Cần đếm theo nhánh.
- Đủ bộ nhớ.

Cân nhắc cách khác

- Chỉ tìm khóa chính xác.
- Có ít truy vấn.
- Chuỗi con / khoảng không theo tiền tố.
- Bảng nhánh quá lớn.

PHẦN VI

06 Bài toán ứng dụng và mở rộng

Đếm tiền tố · Từ thứ k · LCP · Binary Trie · Aho–Corasick

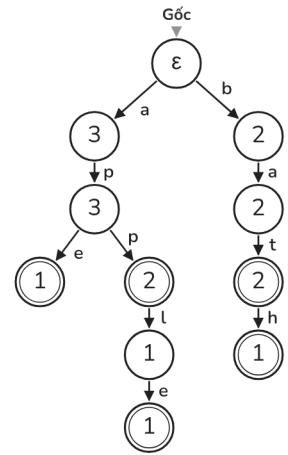
Đếm tiền tố bằng passCount

`app` `apple` `ape` `bat` `bath`

ap → 3 từ

Đến node `ap`, đọc **passCount**.

passCount: số lần xuất hiện của các từ có tiền tố này.



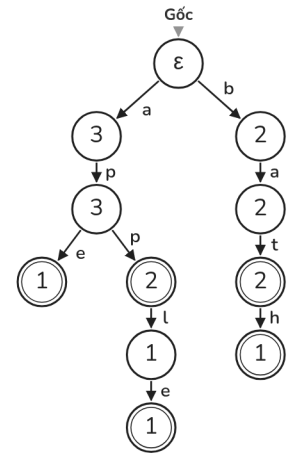
Đếm từ có tiền tố ap.

Số trong node: passCount

Duy trì passCount khi dữ liệu thay đổi

- **Chèn:** tăng bộ đếm trên đường đi.
- **Xóa một từ đang có:** giảm trên đường đi.
- `endCount` chỉ thay đổi ở **node cuối**.

Bộ đếm phản ánh những từ **hiện còn lưu**.



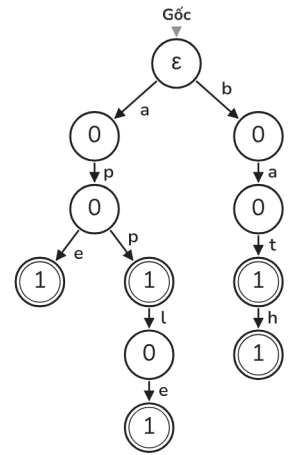
Chèn app, cập nhật bộ đếm.

Số trong node: passCount

Liệt kê theo thứ tự từ điển

- Xét từ kết thúc **trước khi xuống con**.
- Duyệt nhánh theo **a** → **z**.

ape → app → apple → bat → bath



Liệt kê theo thứ tự từ điển.

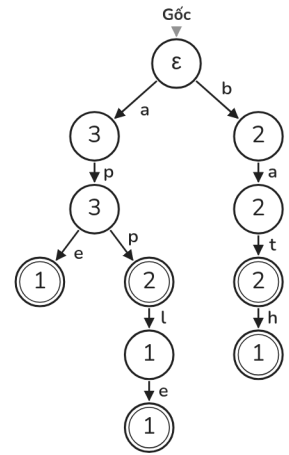
Tìm từ thứ k bằng cách bỏ qua nhánh

Dùng **số từ trong cây con** để quyết định đi vào hay bỏ qua.

k = 3 → apple

Cây con a: 3 từ · Cây con b: 2 từ.

Không cần liệt kê mọi từ trước đó.



Tìm từ thứ 3.

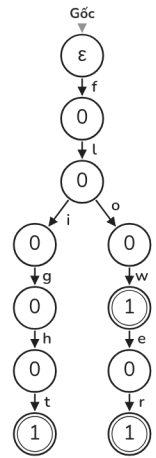
Số trong node: passCount

Tiền tố chung dài nhất

`flower` `flow` `flight`

LCP = fl

Đi tiếp khi mọi từ còn chung cạnh.
Dừng khi **phân nhánh** hoặc **có từ kết thúc**.



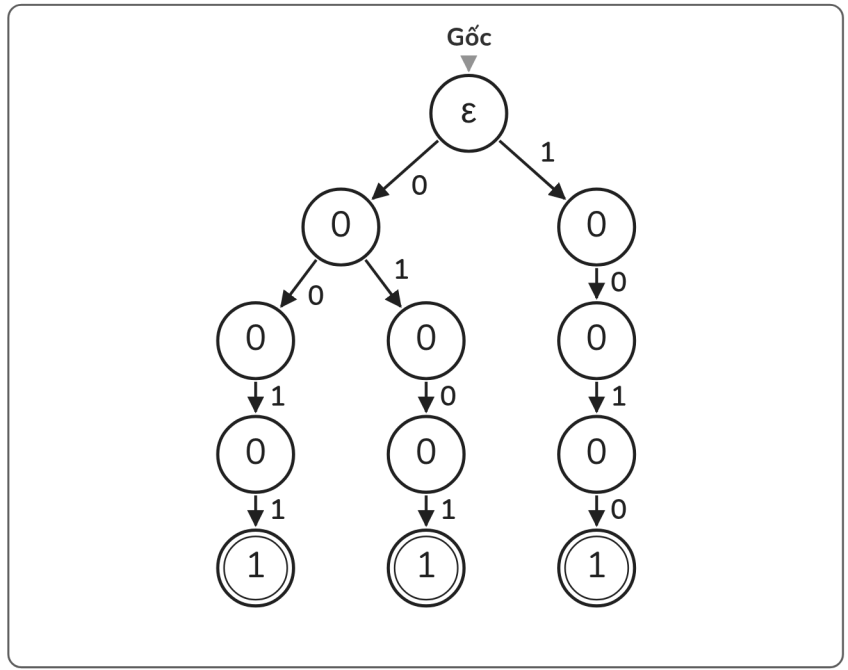
Tìm tiền tố chung dài nhất.

Binary Trie: xem số như một dãy bit

Lưu các số `{3, 5, 10}` bằng **cùng 4 bit**:

Số	Dãy bit
3	0011
5	0101
10	1010

Mỗi cạnh mang **0** hoặc **1**.



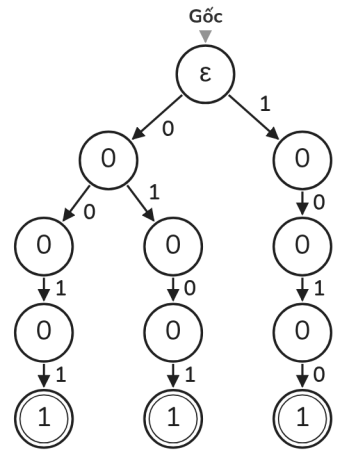
XOR lớn nhất: ưu tiên bit đối nghịch

Với **6 = 0110**, tìm x để x XOR 6 lớn nhất.

Từ bit cao xuống thấp: **chọn bit đối nghịch**; thiếu thì theo nhánh còn lại.

x	x XOR 6
3	5
5	3
10	**12**

Cùng B bit · O(B) / truy vấn.



Tìm XOR lớn nhất với 6.

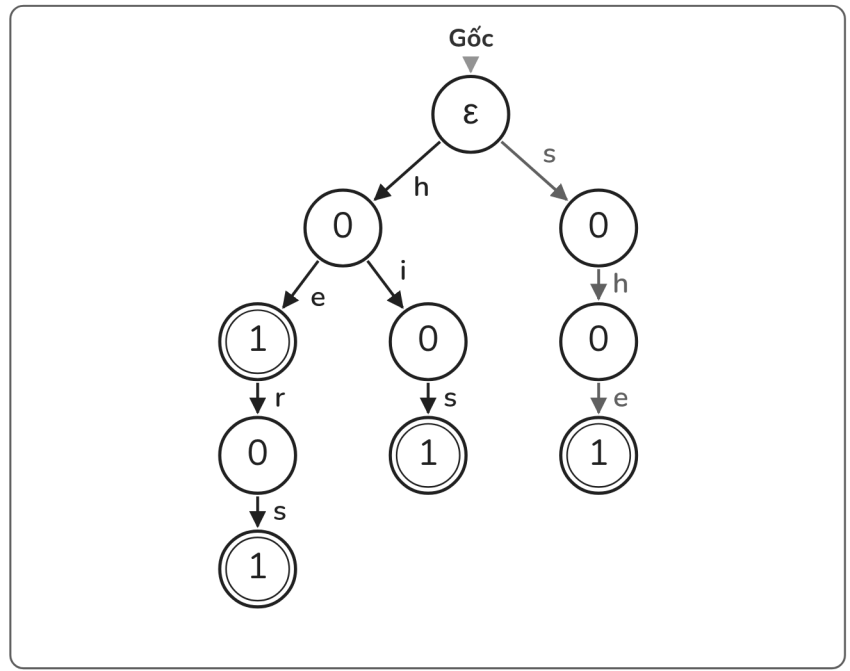
Bài toán: tìm nhiều mẫu trong văn bản

Các mẫu: `he`, `she`, `hers`, `his`.

Tìm mọi vị trí xuất hiện của các mẫu.

`she` chứa cả `she` và `he`.

Làm sao giữ lại phần đã đọc còn hữu ích?

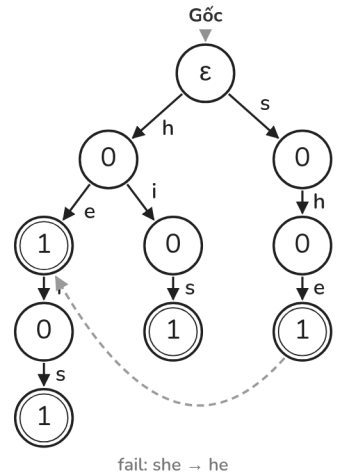


Aho–Corasick: giữ hậu tố bằng failure link

Failure link: hậu tố thực sự dài nhất còn là tiền tố của một mẫu; không có thì về gốc.

`fail("she") = "he"`

Trie + failure link + **thông tin đầu ra** → quét văn bản một lượt, báo cả `she` và `he`.



Quét văn bản “ushers”.

PHẦN VII

07 Ứng dụng trong hệ thống thực tế

Autocomplete · Định tuyến mạng · Các hướng mở rộng

Autocomplete: lấy từ trong cây con

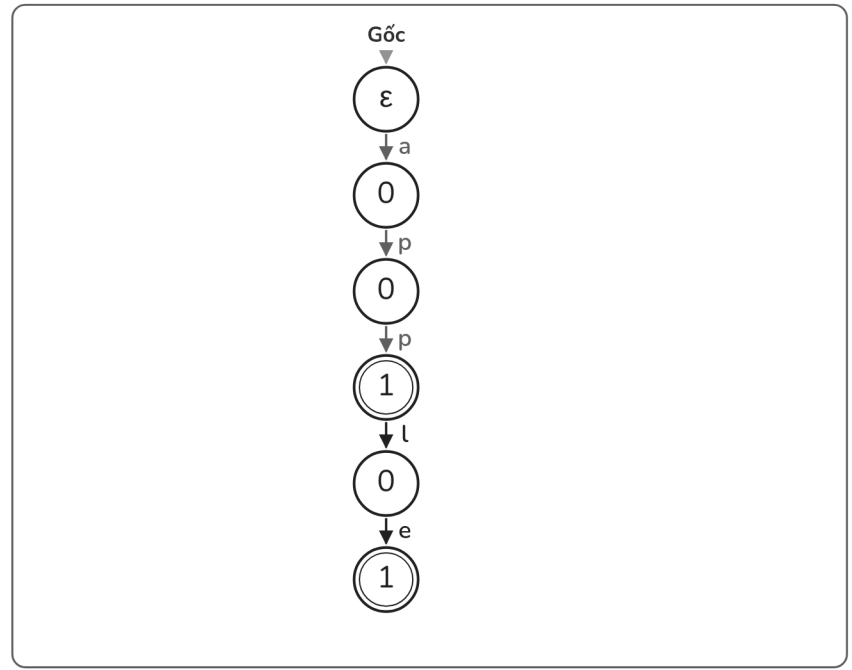
app

app

apple

...

Đến node `app` → duyệt cây con.



Xếp hạng các gợi ý

Thêm **độ phổ biến** để xếp hạng kết quả.

Ngữ cảnh và cá nhân hóa cần thêm cơ chế.

Redis Search dùng compact trie cho gợi ý.

Chọn tuyến mạng: 24 tuyến mẫu

24 tuyến IPv4 · Mỗi tiền tố bit được lưu từ node gốc.

Tuyến	Tiền tố bit	Cổng
0.0.0.0/0	ε	A
10.0.0.0/8	00001010	B
172.16.0.0/12	101011000001	C
192.168.0.0/16	1100000010101000	D
192.168.1.0/24	110000001010100000000001	E
192.168.2.0/24	11000000101010000000010	F
192.168.1.32/27	110000001010100000000001001	G
192.168.1.64/26	11000000101010000000000101	H
10.1.0.0/16	0000101000000001	I
10.1.2.0/24	000010100000000100000010	J
172.16.1.0/24	101011000001000000000001	K
172.20.0.0/16	1010110000010100	L

Tuyến	Tiền tố bit	Cổng
192.168.0.0/17	11000000101010000	M
192.168.0.0/20	110000001010100000000	N
192.168.0.0/22	1100000010101000000000	O
192.168.1.0/25	110000001010100000000000010	P
192.168.1.32/28	1100000010101000000000010010	Q
192.168.1.40/29	11000000101010000000000100101	R
192.168.1.40/30	110000001010100000000001001010	S
192.168.1.40/31	1100000010101000000000010010100	T
192.168.1.42/32	11000000101010000000000100101010	U
192.168.3.0/24	110000001010100000000011	V
192.168.128.0/17	11000000101010001	W
192.168.64.0/18	110000001010100001	X

Tra IP từng bit bằng Binary Trie

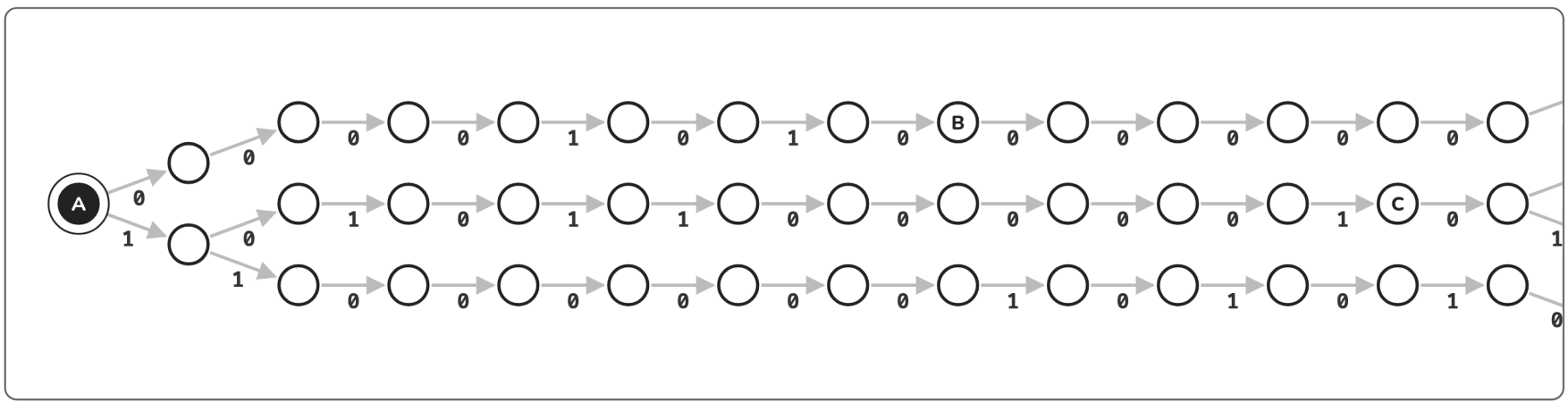
IP đích **192.168.1.42**

24 tuyến · 90 node · Mỗi cạnh = 1 bit

1 1 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0

Đã đi 0 cạnh · Đã xét 0/32 bit

Tốt nhất: **0.0.0.0/0 → A**



Bắt đầu ở gốc: tuyến mặc định → A.

Tìm gần đúng cần thêm khoảng cách sửa

Người dùng gõ sai → khớp tiền tố thuần túy có thể bỏ sót từ.

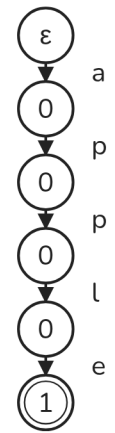
Trie + khoảng cách sửa để tìm gần đúng.

Trie cơ bản chưa tự sửa chính tả.

Nén các đường chỉ có một con

Gộp chuỗi cạnh thành **một nhãn dài hơn**.

Compressed Trie / Radix / Patricia.



Nén đường đi của apple.

PHẦN VIII

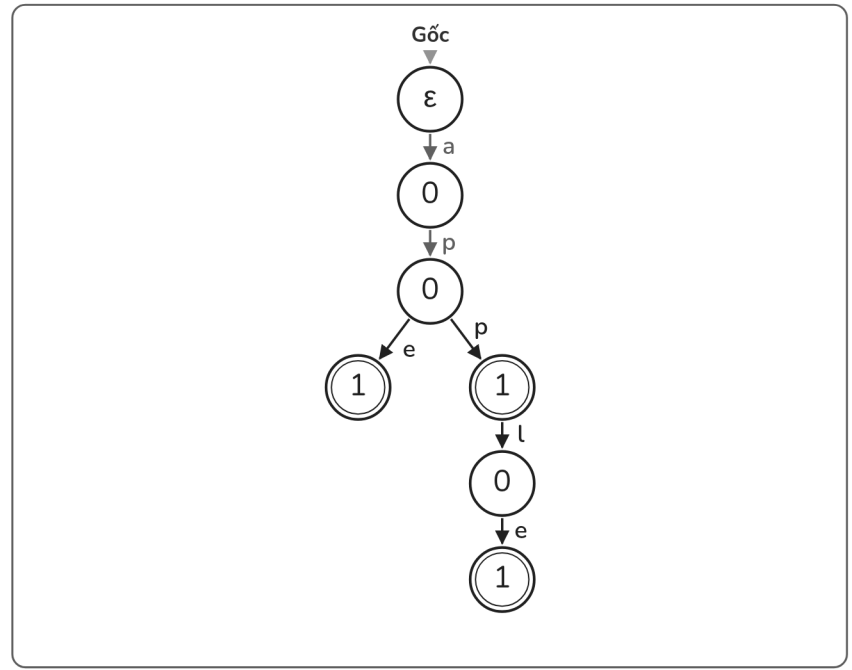
08 Tổng kết

Các ý chính cần nhớ và cơ sở lựa chọn Trie.

Nhớ về cách tổ chức dữ liệu

Từ chung tiền tố dùng chung đường đi.

Node kết thúc phân biệt **từ đã lưu** với **tiền tố**.



Nhớ về chi phí tài nguyên

- Thời gian theo **độ dài khóa**.
- Bộ nhớ theo **số node và bảng chữ cái**.

$$N \le 1 + nL$$

$$N_{\max} = 1 + \sum_{d=1}^{L} \min(n, \sigma^d)$$

Nhớ chọn cấu trúc theo bài toán

Tiền tố là trọng tâm → **cân nhắc Trie**.

Đối chiếu **hashing**, **binary search**, **cây cân bằng**.

Ứng dụng: từ điển · autocomplete · router · Aho–Corasick.

Tài liệu tham khảo

- VNOI Wiki — Trie
- Redis Search
- Aho–Corasick — Tufts
- Lin & Hsieh, 2023 — Tra cứu IP